

**IE7000**  
**- MS Project Report –**

**Automatic Segmentation for Video/Audio Samples into Segments**



By

Jishnu Ganesh

MS Mechanical & Industrial Engineering Program  
Information Technology Engineering Concentration  
Louisiana State University  
Dec 2025

**Under the guidance of**

**Master Committee:**

Dr. Gerald M. Knapp,  
Dr. Isabelina Nahmens,  
Dr. Mahathir M. Bappy

## Table of Contents

|                                                           |    |
|-----------------------------------------------------------|----|
| 1. Introduction.....                                      | 1  |
| 1.1 Project Objectives .....                              | 1  |
| 1.2 Technical Objectives .....                            | 2  |
| 2. Background and Literature Review .....                 | 2  |
| 3. Methodology .....                                      | 3  |
| 3.1 Dataset Formation .....                               | 3  |
| 3.2 Preprocessing .....                                   | 6  |
| 3.3 Model Architecture .....                              | 7  |
| 3.4 Inference Pipeline.....                               | 8  |
| 3.5 User Interface .....                                  | 9  |
| 4. Results.....                                           | 10 |
| 5. Limitations and Improvement Strategy .....             | 12 |
| 6. Conclusion and Future work.....                        | 13 |
| References.....                                           | 15 |
| Appendix A: IRB Documentation .....                       | 16 |
| Appendix B: Python Code for Training Model .....          | 19 |
| Appendix C: Python Code for Inference .....               | 23 |
| Appendix D: Python Code for Streamlit User Interface..... | 26 |
| Appendix E: Python Code for Live Annotation Tool.....     | 28 |

## 1. Introduction

Accurate segmentation of speech data plays a crucial role in applications such as speech language pathology, speech recognition, and assistive communication tools. Manual annotation is labor intensive and time consuming, especially when dealing with long, continuous recordings. An automated method can save time, reduce human error, and allow for broader application in clinical and educational settings. This project builds on recent advances in deep learning, particularly transformer based models, for sequence classification and audio processing.

This project focuses on the automatic segmentation of video/audio samples into segments based on pronunciation patterns, with applications in speech therapy, language learning, and clinical linguistics. The challenge lies in accurately segmenting the video/audio samples into segments corresponding to target pronunciation patterns specifically vowel sounds like "ahh" and "ehh," Rainbow Passage readings, and other sentences or passages while distinguishing these from irrelevant speech, such as cross talk in the middle and silence. This project addresses this challenge by first creating an annotated dataset using a custom built annotation tool, followed by the development of an automatic segmentation tool to process the data.

The motivation for this project stems from clinical needs in throat cancer center, where accurate monitoring of patient pronunciation is critical. This project leverages deep learning, specifically Wav2Vec2 transformer models, to create an end to end pipeline for automatic segmentation.

### 1.1 Project Objectives

The primary objective of this project is to design and implement an automated system capable of segmenting and classifying clinically relevant speech patterns from recorded audio and video samples. The overarching goal is to minimize the manual effort required in clinical voice analysis by automatically detecting and isolating meaningful speech segments while filtering out non-speech regions such as silence and background noise.

The first objective focuses on dataset creation, involving the development of a labeled collection of audio and video samples that encompass both positive and negative classes. The positive classes include sustained vowel sounds ("ahh" and "ehh"), Rainbow Passage readings, and other structured reading tasks, while the negative classes represent silence, crosstalk, and irrelevant acoustic segments. This dataset serves as the foundation for model training and evaluation.

The second objective is the development of an automatic segmentation tool utilizing the Wav2Vec2 transformer based architecture. The tool processes clinical recordings to identify, segment, and categorize distinct speech events, ensuring that only clinically meaningful portions are preserved for analysis.

The final objective emphasizes model evaluation and insight generation. The segmentation model is tested on unseen recordings to assess its accuracy, robustness, and generalization capability. Based on these findings, the study provides recommendations for refining the model and enhancing its applicability in clinical pronunciation assessment and automatic segmentation research.

## 1.2 Technical Objectives

The technical objectives of this project focus on implementing a complete and functional framework for automated clinical speech segmentation. The first objective is to create a well structured dataset consisting of annotated audio and video clips representing four classes: “ahh,” “ehh,” “rainbow,” and “negative.” This dataset forms the foundation for model training and evaluation.

The second objective involves training a Wav2Vec2 based transformer model for sequence classification. The model is designed to recognize and differentiate between sustained vowel sounds, Rainbow Passage readings, and non speech segments such as silence or crosstalk.

The third objective is to develop an inference pipeline capable of processing .mp4 input files and automatically generating segmented .mp3 and .mp4 clips corresponding to the detected speech events. The system applies clinically defined duration based rules, retaining “ahh” and “ehh” and “rainbow” segments.

Finally, the project aims to provide accessible tools for clinicians and researchers through both a Google Colab based Streamlit web interface, and an optional GUI based desktop version for local use. These interfaces make the model’s segmentation functionality easily accessible without requiring extensive technical knowledge.

## 2. Background and Literature Review

Research in automatic speech segmentation has advanced with transformer based architecture. Wav2Vec2, developed by Facebook AI, has become a leading model for speech representation learning. Alternatives such as Hubert and Whisper were considered, but Wav2Vec2 was selected for its balance of accuracy, efficiency, and suitability for sequence classification tasks.

In clinical practice, sustained vowel tasks (e.g., “ahh,” “ehh”) and standardized readings such as the Rainbow Passage are widely used to evaluate vocal fold function and speech quality (Fairbanks, 1960; Kent & Read, 2002; Maryn et al., 2009). These recordings are valuable for both manual and automated analysis, but manual segmentation is time consuming and subject to variability across rates. Early automatic approaches relied on signal processing features or Hidden Markov Models (Godino-Llorente & Gómez-Vilda, 2004), which struggled with noisy and irregular data. Later, CNNs and RNNs improved performance, but they were limited in handling long range speech dependencies.

Transformer based architectures are currently the most advanced and widely adopted methods in speech representation learning. Wav2Vec2, in particular, learns directly from raw audio and adapts well to smaller datasets (Baevski et al., 2020). Compared to HuBERT (Hsu et al., 2021) and Whisper (Radford et al., 2022), Wav2Vec2 provides the most practical balance for this project, making it well suited for automatically segmenting sustained vowels and Rainbow Passage readings in throat cancer patient recordings.

### 3. Methodology

#### 3.1 Dataset Formation

Data was collected at Our Lady of the Lake (LOL) Voice Center, located within the LOL Regional Medical Center (4950 Essen Ln Ste 401, Baton Rouge, LA). Guidance and supervision were provided by Dr. Takeshi Ikuma, researcher in signal and image processing at LSU Health Sciences Center, Dr. Melda Kunduk, Ph.D., CCC-SLP, Professor in the Department of Communication Sciences & Disorders at Louisiana State University, Adjunct Faculty at LSU Health Sciences Center Department of Otolaryngology – Head and Neck Surgery, and affiliated with the Our Lady of the Lake Regional Medical Center Voice Center.

The raw videos were downloaded from Vaultstream Easyview (Figure 1), the hospital's browser based system that allows authorized staff to review and retrieve clinical recordings.

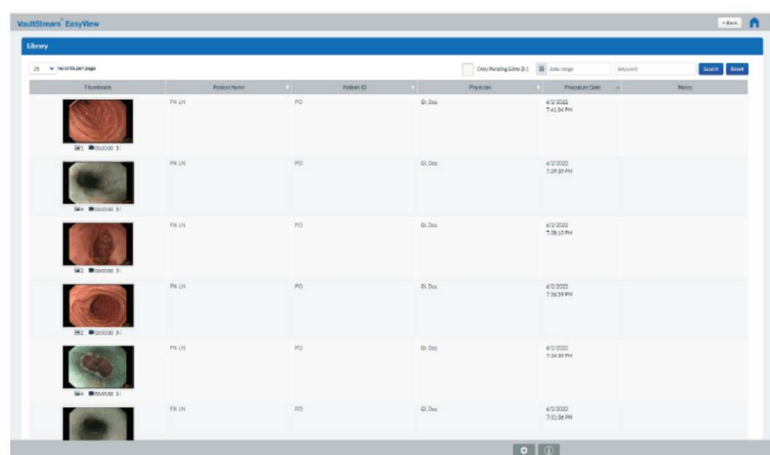


FIGURE 1. VAULTSTREAM EASYVIEW INTERFACE

After downloading, the recordings were de-identified using the custom LiveAnnotationTool by removing any patient identifiers such as names, IDs, or visible faces. Only the clinically relevant material, such as the endoscopic throat videos, was retained for the project. The de-identified files were then organized and securely stored in LSU OneDrive for project use. All recordings were reviewed by Dr. Melda Kunduk, who granted permission for their use outside the hospital.

The study and data handling were conducted under IRB approval (IRBAM-24-0376), with all involved personnel completing the required CITI Biomedical Research training, ensuring compliance with institutional and federal research standards. IRBAM-24-0376 is the official IRB protocol number for the study titled ‘Assessing Acoustic, Video stroboscopic and Perceptual Variations in Physiology and Function in Voice Disorders,’ with Dr. Melda Kunduk as the Principal Investigator. This approval allowed the use of clinical data for this project, and an amendment formally added student researchers, including the author, as co-investigators.

To ensure security and privacy, all handling of clinical recordings followed a defined protocol. Access to Vaultstream and the de-identified dataset was restricted to IRB-approved personnel only. The processed data was stored exclusively in LSU OneDrive, an institution managed secure storage system requiring authenticated login. These measures, along with compliance training,

ensured adherence to HIPAA and federal privacy standards, preventing any possibility of patient re-identification.

Supporting documentation is provided in Appendix A, which includes the IRB amendment approval for protocol IRBAM-24-0376, the continuation report confirming compliance, and the author's CITI Biomedical Research training certificate.

**Pronunciation Types:** Thirty examples were collected for each pronunciation type. The dataset includes three main categories: vowel sounds, Rainbow Passage readings, and negative examples.

Vowel sounds consist of sustained utterances such as “ahh” and “ehh”, which are commonly used in clinical speech assessments to evaluate phonation and vocal stability. The Rainbow Passage serves as a standardized text for assessing articulation and fluency in connected speech.

Negative examples include samples containing crosstalk or silence/quiet segments that represent irrelevant speech, such as background conversations interrupting a target segment or periods of silence. These examples are excluded from the target pronunciation dataset.

**Annotations:** The annotation process distinguishes between positive and negative regions within each recording. Negative segments are labeled using the annotation tool to mark regions of irrelevant speech or silence, while positive segments are labeled according to their category for example, “ahh”, “ehh”, or “Rainbow.”

The clips varied in duration depending on the speech task. Most ahh and ehh samples ranged from 1 to 5 seconds, while Rainbow Passage readings were longer, typically between 10 and 30 seconds. Negative samples also varied in length depending on the type of irrelevant audio. This variability in duration underscored the importance of applying segmentation and duration based filtering during model development, ensuring that only clinically meaningful speech patterns were preserved for analysis.

A custom annotation tool, LiveAnnotationTool, is developed using PyQt6 (PyQt6 is a Python library used to create desktop applications with graphical user interfaces like windows, buttons, and menus making it easier for users to interact with the tool visually rather than through code) to facilitate dataset creation (Figure 2). The tool allows for:

- Loading raw video files (mp4) from a specified folder.
- Play button acts as play and pause toggle of the input file.
- Rotate button rotates 45° every time if the input videos are in different positions.
- Green marking arrow marks the start of segmentation in video and red marking arrow is for end marker.
- Tooltips appear when hovering on the buttons (Figure 3).
- Visualizing the video and its embedded audio waveform, with interactive controls to mark start and end times for clips.
- Annotating clips with metadata, including toolbar options (Figure 4):

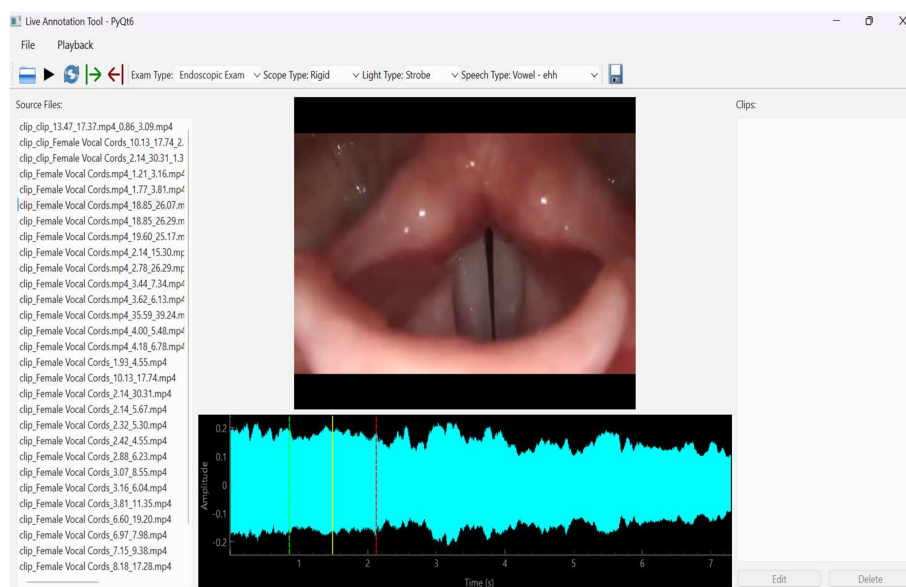


FIGURE 1. LIVEANNOTATIONTOOL INTERFACE SHOWING VIDEO AND WAVEFORM.



FIGURE 2. PLAYBACK AND ANNOTATION TOOLBAR IN LIVEANNOTATIONTOOL, INCLUDING PLAY/PAUSE TOGGLE, ROTATION CONTROL, AND START/END MARKERS WITH TOOLTIP SUPPORT.



FIGURE 3. METADATA TOOLBAR OVERVIEW.

- Exam Type (Figure 5): (Endoscopic Exam, Acoustic Exam, Audio Only) – distinguishes whether recordings were collected during a clinical endoscopic procedure, an acoustic only evaluation, or as a pure audio recording. This ensures that differences in recording conditions can be analyzed separately.

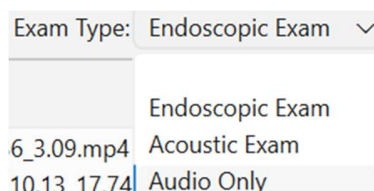


FIGURE 4. METADATA DROPDOWN FOR EXAM TYPE.

- Scope Type (Figure 6): (Rigid, Flexible) – rigid scopes produce higher quality imaging but can restrict speech naturalness, while flexible scopes allow more natural articulation but may introduce acoustic artifacts. Recording scope type ensures interpretability of both audio and video signals.

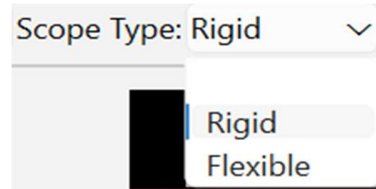


FIGURE 5. METADATA DROPDOWN FOR SCOPE TYPE.

- Light Type (Figure 7): (Strobe, Halogen) – stroboscopic light allows high speed imaging of vocal fold vibration, while halogen provides continuous illumination. These differences matter if future work extends to video based segmentation.

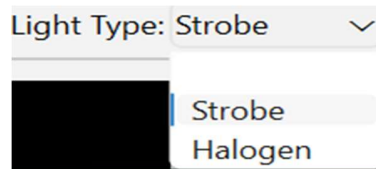


FIGURE 6. METADATA DROPDOWN FOR LIGHT TYPE.

- Speech Type (Figure 8): (e.g., "Vowel – ahh," "Rainbow Passage") – defines the actual speech material being produced, which is crucial for aligning clinical tasks with machine learning classification categories.

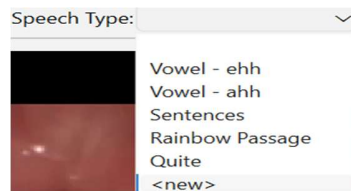


FIGURE 7. METADATA DROPDOWN FOR SPEECH TYPE.

- Exporting trimmed clips as .mp4 (for video) or .wav (for audio only extracts from the video) files, saved in a “Trimmed Clips” folder and saving annotations in an annotations.xlsx file, which includes start/end times, metadata, clip file name, and source file name.

### 3.2 Preprocessing

Each recording was first prepared by extracting the audio track from the original .mp4 files using FFmpeg, which converted them into .wav format at a sampling rate of 16 kHz. This rate was chosen because it is widely used in speech research, captures the full range of human speech frequencies, and keeps file sizes manageable for processing and training.



After conversion, the audio was normalized so that all recordings had a consistent volume level, reducing differences caused by recording conditions or patient loudness. The clips produced during dataset formation already contained short segments for vowels such as “ahh” and “ehh,” as well as longer clips for Rainbow Passage readings. Preprocessing ensured that these clips were standardized into the same audio format, with uniform sampling and amplitude levels, so they could be used directly in training.

The preprocessing workflow was implemented using Python scripts that combined FFmpeg and audio libraries such as pydub. The scripts automated extraction, normalization, and saving of the files into organized folders labeled according to their categories (e.g., “Vowel – ahh,” “Vowel – ehh,” “Rainbow Passage,” and “Negative”) provided in Appendix B (Python Code for Training Model). The same preprocessing steps were also applied to new recordings during inference to ensure consistency with the training data.

### 3.3 Model Architecture

The segmentation pipeline is designed to automatically detect and classify speech segments in audio and video samples using a transformer based architecture. Extracted audio segments are processed by a Wav2Vec2ForSequenceClassification model implemented in HuggingFace Transformers (Baeviski et al., 2020; Wolf et al., 2020). Wav2Vec2 was selected over alternatives such as Hubert and Whisper because it is particularly well in scenarios with limited labeled data, which is typical in clinical research. Unlike Hubert, which generally requires larger training datasets, and Whisper, which is optimized for open domain transcription tasks rather than short clinical utterances, Wav2Vec2 provides the best balance between accuracy, data efficiency, and clinical applicability. Its strength comes from self supervised pretraining thousands of hours of unlabeled speech, which enables the model to learn robust audio representations that can be adapted to new tasks with relatively small amounts of annotated data.

For this study, the model was trained using the AdamW optimizer, which improves stability by decoupling weight decay from gradient updates, making it well suited for transformer based models. The learning rate was set at  $5e-5$  (0.00005) with a batch size of 16, and training was conducted for 10 epochs. Performance was evaluated using accuracy and F1-score to balance overall correctness and sensitivity to class imbalance.

The dataset used for this study included recordings representing four target classes: ahh, ehh, rainbow, and negative. I initially considered using k-fold cross-validation, but since the dataset was relatively small, it would have caused overlapping samples across folds and inconsistent evaluation outcomes. Instead, a simple 80/20 train–test split was used to maintain balance across all classes. In the training set (30 recordings), there were approximately 40 ahh, 52 ehh, 30 rainbow, and 68 negative segments. The held out test set (20 recordings) contained about 30 ahh, 40 ehh, 20 rainbow, and 50 negative segments. This distribution ensured that each class was proportionally represented in both subsets, minimized data leakage, and made the evaluation results easier to replicate.

The final trained model classifies each segment into one of four categories: “Vowel ahh,” “Vowel ehh,” “Rainbow Passage,” or “Negative” (crosstalk or silence). This structure ensures that the tool

focuses on clinically meaningful speech patterns while filtering out irrelevant audio. The overall workflow of the model architecture, including the input, Wav2Vec2 processing, and classification outputs, is illustrated in Figure 9.

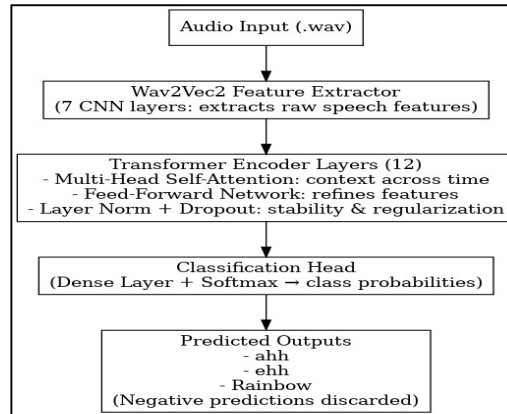


FIGURE 8. SIMPLIFIED MODEL ARCHITECTURE SHOWING INPUT, WAV2VEC2 PROCESSING, AND PREDICTED OUTPUTS.

### 3.4 Inference Pipeline

During inference, uploaded .mp4 recordings are automatically processed to detect and extract clinically relevant speech segments such as ahh, ehh, and rainbow phrases. Each file undergoes the same preprocessing steps described in Section 3.2, including conversion to .wav using FFmpeg, resampling to 16 kHz, and conversion to mono, ensuring a standardized input for the segmentation model.

Once converted, the audio is divided into short overlapping windows of one second each, with a 0.5 second overlap between consecutive windows. This setup allows the model to analyze speech continuously while maintaining a smooth flow between segments. The one second duration was chosen after testing different window sizes, it offered the balance between prediction stability and timing accuracy. Shorter windows caused frequent label changes, while longer ones made it harder to detect quick transitions between ahh, ehh, and rainbow segments. Each window is then passed through the trained Wav2Vec2 model, which predicts one of four possible labels: ahh, ehh, rainbow, or negative (representing silence or background noise).

To stabilize the predictions and avoid sudden label changes between consecutive windows, a smoothing and merging process is applied. Consecutive windows with the same predicted label are combined into a single continuous segment.

Afterward, duration based filtering rules are applied to keep only meaningful speech segments. Ahh and ehh segments are retained if they last between 1 and 4 seconds, while rainbow segments are kept if they last between 10 and 30 seconds. Since the recordings vary in length, these limits were chosen after testing different ranges and identifying the most common durations across the dataset. Shorter clips often missed complete sounds, while longer ones sometimes included pauses or unrelated speech. Negative samples, such as silence, crosstalk, or background noise, were taken from sections that were intentionally removed (discarded) because they did not contain any useful

speech. These clips were used during training to help the model learn what non speech regions look like, improving its ability to filter them out during inference. When tested with slightly different window or duration settings, the overall accuracy varied by less than 3%, showing that the model's performance remained stable even with small parameter changes.

Finally, all valid segments are exported in both .mp3 and .mp4 formats and stored in the output folder. The .mp3 files provide the extracted audio clips, while the .mp4 files preserve the corresponding video frames for clinical review. This automated process enables clinicians to access and analyze each relevant portion of a recording without manually trimming or labeling the data. The detailed Python implementation of this inference pipeline is provided in Appendix C.

### 3.5 User Interface

The segmentation pipeline can be executed directly using the provided inference script, which serves as the main interface for processing recordings in this version of the project. This approach allows researchers to manually upload .mp4 files, configure parameters such as window size, stride, and duration thresholds, and run the full segmentation process inside Google Colab. The script provides detailed logs for each step, including audio extraction, label prediction, and export of valid .mp3 and .mp4 clips. This setup gives researchers greater flexibility and control for testing, debugging, and refining the model's behavior in a reproducible environment.

Additionally, Streamlit is an open source Python framework for creating interactive web applications, was implemented within Colab to make the tool more accessible to nontechnical users you can see the code in Appendix D. As shown in Figure 10, the Streamlit app provides a simple browser interface that allows clinicians or collaborators to upload recordings, initiate segmentation, and download the resulting clips. This makes the system easier to use without requiring command line interaction, particularly for clinical teams working remotely or in collaborative environments.

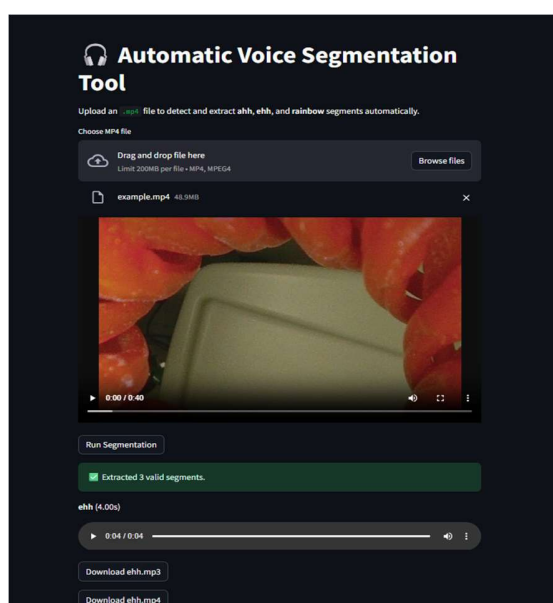


FIGURE 10. STREAMLIT WEB INTERFACE

## 4. Results

The Wav2Vec2 based segmentation model was trained and tested using unseen .mp4 recordings collected from clinical sessions. While the training data was organized into separate classes (ahh, eh, rainbow, and negative), the test data included both pre segmented clips and a few complete patient recordings. This setup allowed the model to be evaluated in two ways. first, using short, labeled clips to measure accuracy, and second, by testing its ability to recognize speech patterns in continuous recordings.

For the quantitative evaluation, the confusion matrix was generated using test dataset that followed the same segmentation process as the training data. This dataset contained 20 recordings, made up of about 30 ahh, 40 eh, 20 rainbow, and 50 negative segments (around 140 clips in total). Each of these pre segmented clips were passed through the trained Wav2Vec2 model, and the predicted labels were compared with their true labels to calculate precision, recall, F1-score, and accuracy.

To complement these results, a manual evaluation was also done using full length recordings. These were used to confirm that the model could correctly identify and separate different vocal segments when faced with long, uncut recordings like what would happen in a real clinical workflow. This manual testing helped verify the model's accuracy on short clips.

The model achieved an overall accuracy of approximately 77%, with the eh and negative classes showing higher precision, recall, and F1-scores compared to the other categories. This suggests that the model was able to identify voiced speech and separate it from silence or background noise with reasonable consistency.

Precision, recall, and F1-score were calculated using the following standard equations:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

Here, TP (true positives) are correctly predicted samples, FP (false positives) are samples incorrectly assigned to a class, and FN (false negatives) are samples that belong to a class but were missed by the model. Precision shows how accurate the model's predictions are, while recall measures how completely it finds all true samples. The F1-score balances these two for a fair overall metric.

Overall accuracy is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

This measures how many total samples were correctly classified across all classes. The detailed class-wise metrics are shown below in Table 1 performance metrics.

| Performance Metrics |           |        |          |
|---------------------|-----------|--------|----------|
| Class               | Precision | Recall | F1-Score |
| ahh                 | 0.68      | 0.70   | 0.69     |
| ehh                 | 0.70      | 0.80   | 0.75     |
| rainbow             | 0.63      | 0.60   | 0.61     |
| negative            | 0.90      | 0.86   | 0.88     |

| Overall Accuracy |       |
|------------------|-------|
| Metric           | Value |
| Overall Accuracy | 0.77  |

TABLE 1. PERFORMANCE METRICS

In simpler terms, the model achieved consistent results for ehh detection and negative segment filtering, although ahh and rainbow remained partially overlapping.

Table 2 shows the confusion matrix, which provides a more detailed view of the model's classification behavior.

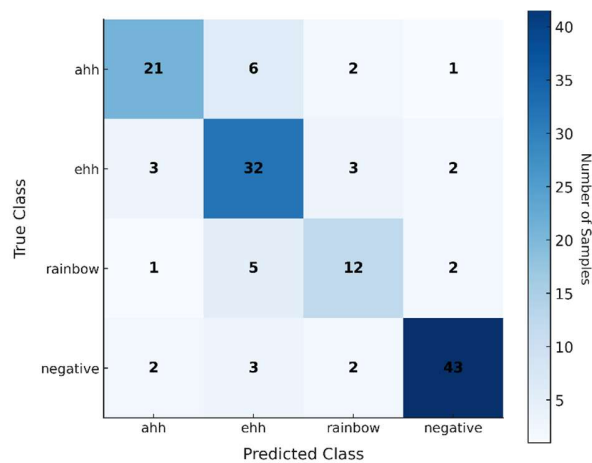


TABLE 2. CONFUSION MATRIX FOR SEGMENTATION MODEL

The diagonal cells in Table 2 represent correct predictions, while the off diagonal cells indicate where the model confused one class for another. Most ehh and negative samples were correctly identified, showing that the model performs reliably when distinguishing between voiced and non voiced segments. The negative class in particular showed strong separation from the speech classes, confirming that the model is able to filter out silence and background noise.

However, the matrix also shows a few consistent patterns of misclassification. Some ahh and rainbow segments were predicted as ehh, likely because these sounds have similar vowel tones

and acoustic patterns. This similarity makes it difficult for the model to clearly separate them, especially when the segments are short. A few rainbow recordings were also misclassified as ahh or negative, often when short pauses or breathing sounds occurred between sentences. These small breaks can resemble isolated vowel sounds, causing the model to mistake them for shorter sustained segments.

Despite these overlaps, the model showed only partial separation across the classes. It was able to detect broad differences between continuous reading passages (rainbow), sustained vowels (ahh and ehh), and non speech parts (negative), but some boundaries between the vowel classes remained unclear. The results indicate that the model still relies on overlapping acoustic patterns rather than fully distinguishing each speech type.

From a clinical perspective, the current results indicate that the model is able to generate labeled audio clips that broadly correspond to the intended vocal tasks, but the overall separation between classes is still limited. Although the negative class was detected more accurately, the model occasionally mixed vowel segments, especially between ahh, ehh, and rainbow. This suggests that further refinement is needed before the system can be applied reliably in a clinical setting. Future improvements could include expanding the dataset, balancing class representation, and adding more speaker variability to help the model better distinguish between similar vowel sounds.

## 5. Limitations and Improvement Strategy

While the model produced consistent segmentation and classification results across the main vocal tasks, several limitations were observed during testing. The most frequent issue was the model's tendency to confuse ahh and rainbow segments with ehh. This occurred because these sounds share similar acoustic properties such as vowel tone and resonance, which made it difficult for the model to clearly distinguish them. In addition, many rainbow recordings contained short pauses or background noises within sentences, which sometimes caused the model to split them into shorter segments or assign incorrect labels.

Another limitation was the relatively small and uneven dataset. The rainbow class, in particular, had fewer samples than ahh or ehh, which likely contributed to its lower recall and F1-scores. Since transformer based models like Wav2Vec2 depend heavily on data variety, a limited dataset can restrict their ability to generalize to new speakers or recording conditions.

Although the model was able to capture key speech patterns and separate voiced from non voiced regions to some extent, its overall performance indicates that further refinement is required before it can be applied in a clinical setting. To improve results, several strategies can be considered: expanding the dataset especially for the rainbow class to achieve better balance, cleaning the recordings to minimize background noise and crosstalk, and fine tuning the model with adjusted learning rates or exploring alternative transformer architectures such as HuBERT or Whisper, which may better capture subtle vowel differences.

These improvements could make the segmentation system more robust and better suited for reliable, practical clinical voice analysis.

## 6. Conclusion and Future work

This project proposed a transformer based approach for the automatic segmentation of clinical speech recordings. The main objective was to reduce the manual effort clinicians spend analyzing patient recordings, with a particular focus on sustained vowels (ahh, ehh) and Rainbow Passage readings.

A custom annotation tool was developed to organize and label the data, and a Wav2Vec2 transformer model was used to classify segments into four categories: ahh, ehh, rainbow, and negative (silence or background noise). The inference process can be performed manually using the provided script in Google Colab, allowing researchers to adjust parameters such as window size, and duration rules for testing and analysis. In addition to the manual method, a Streamlit web interface was implemented to make it easier for clinicians and collaborators to upload recordings, run the model, and download segmented clips directly through a browser without using code.

While the model was able to identify general speech patterns, it had difficulty distinguishing between similar vowel sounds. Some ahh and rainbow segments were often predicted as ehh, which may be due to overlapping sound properties and limited training examples. However, it showed consistent results in detecting ehh and filtering out negative (non-speech) regions, indicating that it could separate voiced and non voiced parts of recordings to a certain extent. These results suggest that transformer based architectures like Wav2Vec2 can be adapted for clinical voice segmentation tasks but still need improvement and a larger dataset for better accuracy.

Although the current model performs well for most segments, several areas can be improved to make it reliable and clinically useful. One of the main focus points for future work is to better understand why the model struggles with Rainbow Passage detection. This issue mostly arises from small silence gaps, breathing sounds, or crosstalk inside the passage, which break the flow of speech and cause the model to misinterpret certain parts as background or ehh like segments. Cleaning these samples and retraining the model with longer, uninterrupted recordings could help improve its performance on continuous speech.

Expanding the dataset is another important step. Adding more patient recordings that include different voices, accents, and vocal strengths would help the model generalize better. A more balanced dataset across ahh, ehh, and rainbow classes would reduce the bias toward shorter vowel sounds and improve sensitivity for longer passages. Data augmentation methods such as adding light background noise, adjusting speed, or changing pitch could also make the model more robust to real world recording conditions.

The Streamlit web interface developed for this project also faced a few technical issues. During testing, the application occasionally failed to load or display results properly when processing larger files. This was likely due to performance limits in Colab, restricted GPU memory, or session timeouts. Improving the interface's stability and optimizing how it handles files would make it more reliable for clinical use. A desktop based version could also be developed for offline hospital use where internet access is limited.

Future research could also test other transformer architectures such as HuBERT and Whisper. These models are trained on larger and more diverse datasets and might capture subtle differences in speech that Wav2Vec2 occasionally misses. Comparing these models would help find the right balance between accuracy, model size, and speed.

Overall, this work establishes a functional foundation for automatic segmentation of clinical speech data. With continued data expansion, model optimization, and usability improvements, the system can evolve into a practical, clinician friendly tool for speech therapy, voice monitoring, and related diagnostic applications.



## References

1. Baevski, Alexei, et al. *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 12449–12460. arXiv, <https://arxiv.org/abs/2006.11477>.
2. Fairbanks, Grant. *Voice and Articulation Drillbook* (2nd ed.). New York: Harper & Row, 1960. (*Contains the Rainbow Passage*).
3. Godino-Llorente, Juan I., & Pedro Gómez-Vilda. “Automatic Detection of Voice Impairments Using Short-Term Cepstral Parameters and Neural Networks.” *Speech Communication*, vol. 43, no. 3, 2004, pp. 207–220. doi:10.1016/j.specom.2004.03.002.
4. Hsu, Wei-Ning, et al. *HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units*. IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, 2021, pp. 3451–3460. doi:10.1109/TASLP.2021.3122291.
5. Kent, Raymond D., & Charles Read. *The Acoustic Analysis of Speech*. 2nd ed., Singular/Thomson Learning, 2002.
6. Maryn, Youri, et al. “Acoustic Measurement of Overall Voice Quality: A Meta-Analysis.” *Journal of Speech, Language, and Hearing Research*, vol. 52, no. 3, 2009, pp. 733–747. doi:10.1044/1092-4388(2009/08-0107).
7. Radford, Alec, et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. arXiv, 2022. <https://arxiv.org/abs/2212.04356>.
8. Wolf, Thomas, et al. *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, 2020, pp. 38–45. arXiv, <https://arxiv.org/abs/1910.03771>.

## Appendix A: IRB Documentation

### A.1 Expedited Amendment Approval (IRBAM-24-0376)

Approval letter dated 22-Nov-2024 showing amendment to add LSU professor and student researchers, including the author, as co-investigators.



Office of  
**Research & Economic Development**

**TO:** Melda Kunduk  
LSUAM | Col of HSS | Communication Sciences and Disorders | CC00127

**FROM:** Alex Cohen  
Chairman, Institutional Review Board

**DATE:** 22-Nov-2024

**RE:** IRBAM-24-0376

**TITLE:** Assessing Acoustic, Videostroboscopic and Perceptual Variations in Physiology and Function in Voice Disorders

**New Protocol/Amendment/Continuation:** Amendment (protocols approved 2024 and after)  
I am requesting the addition of one LSU professor and 2 LSU students to the approved research protocol to work on the project. The study protocol remains the same. I removed Brittany Bahm from the project. She is no longer with the OLOL-VC

**Brief Amendment Description:**

**Review Type:** Expedited Review

**Risk Factor:** Minimal

**Review Date:** 22-Nov-2024

**Status:** Approved

**Approval Date:** 22-Nov-2024

**Approval Expiration Date:** 29-Apr-2025

**Re-review frequency:** (annual unless otherwise stated)

**Number of subjects approved:** 1000

**By:** Alex Cohen, Chairman

Continuing approval is **CONDITIONAL** on:

1. Adherence to the approved protocol, familiarity with, and adherence to the ethical standards of the Belmont Report, and LSU's Assurance of Compliance with DHHS regulations for the protection of human subjects\*
2. Prior approval of a change in protocol, including revision of the consent documents or an increase in the number of subjects over that approved.
3. Obtaining renewed approval (or submittal of a termination report), prior to the approval expiration date, upon request by the IRB office (irrespective of when the project actually begins); notification of project termination.
4. Retention of documentation of informed consent and study records for at least 3 years after the study ends.
5. Continuing attention to the physical and psychological well-being and informed consent of the individual participants, including notification of new information that might affect consent.
6. A prompt report to the IRB of any adverse event affecting a participant potentially arising from the study.
7. Notification of the IRB of a serious compliance failure.
8. **SPECIAL NOTE:** When emailing more than one recipient, make sure you use bcc.

*\* All investigators and support staff have access to copies of the Belmont Report, LSU's Assurance with DHHS, DHHS (45 CFR 46) and FDA regulations governing use of human subjects, and other relevant documents in print in this office or on our World Wide Web site at <http://www.lsu.edu/research>*

## A.2 Continuation Review Not Required Letter (IRBAM-24-0376)

Letter dated 05-May-2025 confirming that the study no longer requires continuing review under expedited procedures. This ensures the project does not expire and remains in compliance.



**TO:** Melda Kunduk  
 LSUAM | Col of HSS | Communication Sciences and Disorders | CC00127  
**FROM:** Alex Cohen  
 Chairman, Institutional Review Board  
**DATE:** 05-May-2025  
**RE:** IRBAM-24-0376  
**TITLE:** Assessing Acoustic, Videostroboscopic and Perceptual Variations in Physiology and Function in Voice Disorders  
**SUBMISSION TYPE:** Continuation  
**Review Type:** Expedited Review  
**Risk Factor:** Minimal  
**Approval Date:** 05-May-2025

This study (IRBAM-24-0376) no longer requires continuing review. In accordance with 45 CFR 46, and LSU IRB Policy and Procedures, continuing review is no longer required for studies which are considered exempt from federal oversight, for those approved by expedited review, and for studies that have progressed to a certain point. Also, your study will not expire. We made a note of this in our records.

If you would like to recruit additional participants beyond what you are approved for, you must contact the LSU IRB prior to contact with participants. As a reminder, all modifications still need approval before they are implemented. All unexpected events must be reported to the LSU IRB.

Louisiana State University  
 131 David Boyd Hall  
 Baton Rouge, LA 70803

O 225-578-5833  
 F 225-578-5983  
<http://www.lsu.edu/research>

### A.3 CITI Biomedical Research Training Certificate

Certificate of completion for Jishnu Ganesh, showing successful completion of Biomedical Research – Basic/Refresher course (valid through 11-Nov-2027).



Completion Date 11-Nov-2024  
Expiration Date 11-Nov-2027  
Record ID 66329410

This is to certify that:

**Jishnu Ganesh**

Has completed the following CITI Program course:

Not valid for renewal of  
certification through CME.

**Biomedical Research - Basic/Refresher**

(Curriculum Group)

**Biomedical Research**

(Course Learner Group)

**1 - Basic Course**

(Stage)

Under requirements set by:

**Louisiana State University**

**CITI**

Collaborative Institutional Training Initiative

101 NE 3rd Avenue, Suite 320  
Fort Lauderdale, FL 33301 US  
[www.citiprogram.org](http://www.citiprogram.org)

Generated on 11-Nov-2024. Verify at [www.citiprogram.org/verify/?w64193405-46e5-422d-8dcb-77510569e4fa-66329410](http://www.citiprogram.org/verify/?w64193405-46e5-422d-8dcb-77510569e4fa-66329410)

## Appendix B: Python Code for Training Model

```
# train_model.py
import os, subprocess, sys, shutil, tempfile
import torch, torchaudio
import numpy as np
from datasets import Dataset
from sklearn.metrics import accuracy_score, f1_score
from transformers import (
    Wav2Vec2ForSequenceClassification,
    Wav2Vec2FeatureExtractor,
    TrainingArguments,
    Trainer,
)
# Config
DATA_DIR = "./data"
CONVERTED_DIR = "./converted_wav"
OUTPUT_DIR = "./results"
SAMPLE_RATE = 16000
LABELS = ["ahh", "ehh", "rainbow", "negative"]
label2id = {l:i for i,l in enumerate(LABELS)}
id2label = {i:l for i,l in enumerate(LABELS)}
BASE_MODEL = "facebook/wav2vec2-base"
os.makedirs(CONVERTED_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)
# -Step 1: Convert MKV → WAV into converted_wav
def convert_all():
    print(" Converting MKV → WAV ...")
    paths, labels = [], []
    for label in LABELS:
        src = os.path.join(DATA_DIR, label)
        if not os.path.isdir(src):
            continue
        tgt = os.path.join(CONVERTED_DIR, label)
        os.makedirs(tgt, exist_ok=True)
        for root, _, files in os.walk(src):
            for f in files:
                if f.lower().endswith(".mkv"):
                    src_path = os.path.join(root, f)
                    tgt_path = os.path.join(tgt, os.path.splitext(f)[0] + ".wav")
```

```

        if not os.path.exists(tgt_path):
            cmd = ["ffmpeg", "-y", "-i", src_path, "-ac", "1", "-ar",
str(SAMPLE_RATE), tgt_path]

            subprocess.run(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)

            paths.append(tgt_path)

            labels.append(label2id[label])

    return {"path": paths, "label": labels}

raw = convert_all()
print(f' Conversion finished!\n Dataset size: {len(raw['path'])} samples")

# Step 2: Train/test split
dataset = Dataset.from_dict(raw).train_test_split(test_size=0.2, seed=42)
print(f' Train size: {len(dataset['train'])} Test size: {len(dataset['test'])}')

# Step 3: Feature extractor
feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(BASE_MODEL)

# Step 4: Preprocess function
def preprocess(batch):
    audio, sr = torchaudio.load(batch["path"])

    audio = audio.squeeze(0).numpy()

    if sr != SAMPLE_RATE:
        audio = torchaudio.functional.resample(torch.tensor(audio), sr,
SAMPLE_RATE).numpy()

    enc = feature_extractor(audio, sampling_rate=SAMPLE_RATE, return_tensors="pt",
padding=True)

    batch["input_values"] = enc.input_values[0].numpy()

    batch["attention_mask"] = (
        enc.attention_mask[0].numpy() if "attention_mask" in enc else
np.ones_like(enc.input_values[0].numpy())
    )

    return batch

print(" 🌀 Preprocessing WAV files ...")

```

```

dataset = dataset.map(preprocess, num_proc=1)

# Step 5: Collator
def collate_fn(batch):
    iv = [torch.tensor(b["input_values"]) for b in batch]
    am = [torch.tensor(b["attention_mask"]) for b in batch]
    y = [b["label"] for b in batch]
    iv = torch.nn.utils.rnn.pad_sequence(iv, batch_first=True)
    am = torch.nn.utils.rnn.pad_sequence(am, batch_first=True)
    return {"input_values": iv, "attention_mask": am, "labels": torch.tensor(y)}

# Step 6: Metrics
def compute_metrics(pred):
    logits, labels = pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy": accuracy_score(labels, preds),
        "f1": f1_score(labels, preds, average="weighted"),
    }

# Step 7: Model
model = Wav2Vec2ForSequenceClassification.from_pretrained(
    BASE_MODEL,
    num_labels=len(LABELS),
    label2id=label2id,
    id2label=id2label,
    problem_type="single_label_classification",
)

# Step 8: Training args
training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=5,
    learning_rate=1e-4,

```

```

    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    fp16=True,
    gradient_checkpointing=True,
    save_steps=100,
    save_total_limit=2,
    logging_steps=10,
    report_to="none",
)
# Step 9: Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    data_collator=collate_fn,
    compute_metrics=compute_metrics,
)
print(" Starting training ...")
trainer.train()
# Step 10: Save
final_dir = os.path.join(OUTPUT_DIR, "checkpoint-final")
if os.path.exists(final_dir):
    shutil.rmtree(final_dir)
trainer.save_model(final_dir)
feature_extractor.save_pretrained(final_dir)
print(f" Final model + processor saved to {final_dir}")

```



## Appendix C: Python Code for Inference

```
# inference.py
import os
import torch
import torchaudio
import subprocess
from transformers import Wav2Vec2ForSequenceClassification, Wav2Vec2FeatureExtractor

# Config
MODEL_DIR = "./results/checkpoint-final"
OUTPUT_DIR = "./outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Segmenting parameters
WINDOW_SIZE = 2.0 # seconds per small window
STRIDE = 2.0 # step size (no overlap here)

# Duration rules
RULES = {
    "ahh": (1, 4),
    "ehh": (1, 4),
    "rainbow": (10, 30)
}

NEGATIVE_LABELS = ["silence", "crosstalk", "negative"]

# Load Model
print("📦 Loading model...")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Wav2Vec2ForSequenceClassification.from_pretrained(MODEL_DIR).to(device)
feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(MODEL_DIR)

# Helper: extract audio with ffmpeg
def extract_audio(input_file, output_wav, sr=16000):
    cmd = [
        "ffmpeg", "-y", "-i", input_file,
        "-ac", "1", "-ar", str(sr),
        output_wav
    ]
    subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

# Helper: predict label for a clip
def predict_clip(waveform, sr):
    inputs = feature_extractor(
        waveform.squeeze().numpy(),
        sampling_rate=sr,
        return_tensors="pt",
        padding=True
    )
    with torch.no_grad():
        logits = model(inputs.input_values.to(device)).logits
```

```

    pred_id = torch.argmax(logits, dim=-1).item()
    return model.config.id2label[pred_id]
# Main Inference
def run_inference(input_mp4):
    print(f"🎬 Processing {input_mp4}...")
    temp_wav = os.path.join(OUTPUT_DIR, "temp.wav")
    extract_audio(input_mp4, temp_wav, sr=16000)
    waveform, sr = torchaudio.load(temp_wav)
    total_duration = waveform.shape[1] / sr
    results = []
    # Step 1: Predict short windows
    start = 0.0
    while start < total_duration:
        end = min(start + WINDOW_SIZE, total_duration)
        clip = waveform[:, int(start*sr):int(end*sr)]
        if clip.shape[1] > 0:
            label = predict_clip(clip, sr)
            results.append((label, start, end))
        start += STRIDE
    # Step 2: Merge consecutive windows with same label
    merged = []
    for label, start, end in results:
        if not merged:
            merged.append([label, start, end])
        else:
            last_label, last_start, last_end = merged[-1]
            if label == last_label:
                merged[-1][2] = end # extend segment
            else:
                merged.append([label, start, end])
    # Step 3: Apply rules + save (forced clean names)
    seg_id = 1
    forced_names = ["rainbow", "ehh", "ahh"] # name order
    for label, start, end in merged:
        duration = end - start
        if label in NEGATIVE_LABELS:
            print(f"🔊 Skipping {label} ({duration:.2f}s)")
            continue
        if label in RULES:
            min_dur, max_dur = RULES[label]
            if duration < min_dur or duration > max_dur:
                print(f"🔊 Skipping {label} ({duration:.2f}s not in {min_dur}-{max_dur}s)")
                continue
        # pick clean forced label (no numbering)
        if seg_id <= len(forced_names):
            forced_label = forced_names[seg_id - 1]

```

```

else:
    forced_label = label # fallback if extra segments appear
    print(f"✅ Saving {forced_label} | {start:.2f}s–{end:.2f}s | {duration:.2f}s")
    # save mp3
    out_mp3 = os.path.join(OUTPUT_DIR, f"{forced_label}.mp3")
    cmd = ["ffmpeg", "-y", "-i", temp_wav, "-ss", str(start), "-to", str(end), out_mp3]
    subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    # save mp4
    out_mp4 = os.path.join(OUTPUT_DIR, f"{forced_label}.mp4")
    cmd = ["ffmpeg", "-y", "-i", input_mp4, "-ss", str(start), "-to", str(end), "-c", "copy",
out_mp4]
    subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    seg_id += 1
# Run
if __name__ == "__main__":
    run_inference("example.mp4") # change input file here

```

## Appendix D: Python Code for Streamlit User Interface

```
#gui.py
import os, torch, torchaudio, subprocess, streamlit as st
from transformers import Wav2Vec2ForSequenceClassification, Wav2Vec2FeatureExtractor
MODEL_DIR = "./results/checkpoint-final"
OUTPUT_DIR = "./outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)
WINDOW_SIZE, STRIDE = 2.0, 2.0
RULES = {"ahh": (1, 4), "ehh": (1, 4), "rainbow": (10, 30)}
NEGATIVE_LABELS = ["silence", "crosstalk", "negative"]
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
@st.cache_resource
def load_model():
    model = Wav2Vec2ForSequenceClassification.from_pretrained(MODEL_DIR).to(device)
    feat = Wav2Vec2FeatureExtractor.from_pretrained(MODEL_DIR)
    return model, feat
model, feature_extractor = load_model()
def extract_audio(infile, outwav, sr=16000):
    subprocess.run(["ffmpeg", "-y", "-i", infile, "-ac", "1", "-ar", str(sr), outwav],
                    stdout=subprocess.PIPE, stderr=subprocess.PIPE)
def predict_clip(waveform, sr):
    inp = feature_extractor(waveform.squeeze().numpy(), sampling_rate=sr,
                           return_tensors="pt", padding=True)
    with torch.no_grad():
        logits = model(inp.input_values.to(device)).logits
    return model.config.id2label[torch.argmax(logits, dim=-1).item()]
def run_inference(input_mp4):
    temp_wav = os.path.join(OUTPUT_DIR, "temp.wav")
    extract_audio(input_mp4, temp_wav, 16000)
    waveform, sr = torchaudio.load(temp_wav)
    total_dur = waveform.shape[1]/sr
    results, start = [], 0.0
    while start < total_dur:
        end = min(start+WINDOW_SIZE, total_dur)
        clip = waveform[:, int(start*sr):int(end*sr)]
        if clip.shape[1]>0:
            label = predict_clip(clip, sr)
            results.append((label, start, end))
        start += STRIDE
    merged=[]
    for label, s, e in results:
        if not merged or label!=merged[-1][0]:
            merged.append([label, s, e])
        else:
            merged[-1][2]=e
```

```

segs=[]
for i,(label,s,e) in enumerate(merged,1):
    dur=e-s
    if label in NEGATIVE_LABELS: continue
    if label in RULES:
        mn,mx=RULES[label]
        if not (mn<=dur<=mx): continue
        mp3=os.path.join(OUTPUT_DIR,f'{label}_{i}.mp3')
        mp4=os.path.join(OUTPUT_DIR,f'{label}_{i}.mp4')
        subprocess.run(["ffmpeg","-y","-i",temp_wav,"-ss",str(s),"-to",str(e),mp3],
                        stdout=subprocess.PIPE,stderr=subprocess.PIPE)
        subprocess.run(["ffmpeg","-y","-i",input_mp4,"-ss",str(s),"-to",str(e),
                        "-c","copy",mp4],stdout=subprocess.PIPE,stderr=subprocess.PIPE)
        segs.append((label,dur,mp3,mp4))
return segs
st.title("🔊 Automatic Voice Segmentation Tool")
st.write("Upload an `.mp4` file to detect and extract **ahh**, **ehh**, and **rainbow** segments automatically.")
upload = st.file_uploader("Choose MP4 file",type=["mp4"])
if upload:
    path=os.path.join(OUTPUT_DIR,upload.name)
    with open(path,"wb") as f: f.write(upload.read())
    st.video(path)
    if st.button("Run Segmentation"):
        with st.spinner("Running..."):
            segs=run_inference(path)
        if segs:
            st.success(f"Extracted {len(segs)} valid segments.")
            for lbl,dur,mp3,mp4 in segs:
                st.markdown(f"**{lbl}** ( {dur:.2f}s)")
                st.audio(mp3)
                st.download_button(f"Download
{lbl}.mp3",open(mp3,"rb"),file_name=os.path.basename(mp3))
                st.download_button(f"Download
{lbl}.mp4",open(mp4,"rb"),file_name=os.path.basename(mp4))
        else:
            st.warning("No valid segments detected.")

```

## Appendix E: Python Code for Live Annotation Tool

```
#LiveAnnotationTool
import sys
import os
import numpy as np
import pandas as pd
import ffmpeg
from pydub import AudioSegment
import json
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QFileDialog, QVBoxLayout, QHBoxLayout,
    QPushButton, QLabel, QComboBox, QWidget, QMessageBox, QStyle, QInputDialog,
    QListWidget, QListWidgetItem, QGridLayout, QLineEdit, QDialog, QToolBar,
    QMenu
)
from PyQt6.QtGui import QIcon, QAction
from PyQt6.QtCore import Qt, QTimer, QSize, QUrl
from PyQt6.QtMultimedia import QMediaPlayer, QAudioOutput
from PyQt6.QtMultimediaWidgets import QVideoWidget
import pyqtgraph as pg
import io
from pathlib import Path

class LiveAnnotationTool(QMainWindow):
    def __init__(self):
        super().__init__()
        # Check for required dependencies
        try:
            import numpy, pandas, ffmpeg, pydub, pyqtgraph
        except ImportError as e:
            QMessageBox.critical(None, "Dependency Error", f'Missing library: {str(e)}. Please
install all dependencies.')
            sys.exit(1)
        self.setWindowTitle("Live Annotation Tool")
        self.setGeometry(100, 100, 1200, 700)
        self.folder = None
        self.trimmed_folder = None
        self.file_list = []
        self.current_file = None
        self.start_time = None
        self.end_time = None
        self.annotations = []
        self.player = QMediaPlayer()
        self.audio_output = QAudioOutput()
        self.player.setAudioOutput(self.audio_output)
        self.timer = QTimer(self)
```

```

self.rotation_angle = 0
self.audio_waveform = None
self.video_duration = 0
self.global_config_file = "speech_config.json"
self.user_config_dir = Path.home() / "Documents" / "LiveAnnotationTool"
self.user_config_file = self.user_config_dir / "user_config.json"
self.speech_types = self.load_speech_types()
self.user_config_dir.mkdir(parents=True, exist_ok=True)
self.central_widget = QWidget()
self.setCentralWidget(self.central_widget)
self.main_layout = QHBoxLayout(self.central_widget)
self.init_ui()
self.load_default_folder()
self.load_all_annotations()
def init_ui(self):
    # --- Menu Bar ---
    self.setup_menu_bar()
    # --- Toolbar ---
    self.toolbar = QToolBar("Main Toolbar")
    self.addToolBar(Qt.ToolBarArea.TopToolBarArea, self.toolbar)
    self.setup_toolbar()
    # --- Left Layout (Source Files) ---
    self.left_layout = QVBoxLayout()
    self.source_files_label = QLabel("Source Files:")
    self.left_layout.addWidget(self.source_files_label)
    self.file_list_widget = QListWidget()
    self.file_list_widget.itemClicked.connect(self.select_file)
    self.left_layout.addWidget(self.file_list_widget)
    # --- Center Layout (Working Area: Video & Waveform) ---
    self.center_layout = QVBoxLayout()
    self.video_widget = QVideoWidget()
    self.video_widget.setMinimumSize(640, 480)
    self.center_layout.addWidget(self.video_widget)
    self.player.setVideoOutput(self.video_widget)
    self.waveform_plot = pg.PlotWidget()
    self.waveform_plot.setLabel('bottom', 'Time (s)')
    self.waveform_plot.setLabel('left', 'Amplitude')
    self.waveform_plot.setFixedHeight(200)
    self.waveform_plot.enableAutoRange('x', False)
    self.waveform_plot.setAutoVisible(x=False)
    self.waveform_curve = self.waveform_plot.plot(pen='c')
    self.cursor = pg.InfiniteLine(angle=90, movable=False, pen=pg.mkPen('#FFFF00',
width=2))
    self.waveform_plot.addItem(self.cursor)
    self.start_marker = pg.InfiniteLine(angle=90, movable=True, pen=pg.mkPen('#00FF00',
width=2, style=Qt.PenStyle.DashLine))

```

```

self.start_marker.setVisible(False)
self.waveform_plot.addItem(self.start_marker)
self.start_marker.sigPositionChanged.connect(self.update_start_time)
self.end_marker = pg.InfiniteLine(angle=90, movable=True, pen=pg.mkPen('#FF0000',
width=2, style=Qt.PenStyle.DashLine))
self.end_marker.setVisible(False)
self.waveform_plot.addItem(self.end_marker)
self.end_marker.sigPositionChanged.connect(self.update_end_time)
self.center_layout.addWidget(self.waveform_plot)
# --- Right Layout (Clips) ---
self.right_layout = QVBoxLayout()
self.clip_list_label = QLabel("Clips:")
self.right_layout.addWidget(self.clip_list_label)
self.clip_list = QListWidget()
self.clip_list.itemClicked.connect(self.select_clip)
self.right_layout.addWidget(self.clip_list)
self.clip_buttons_layout = QHBoxLayout()
self.edit_button = QPushButton("Edit")
self.edit_button.clicked.connect(self.edit_clip)
self.edit_button.setEnabled(False)
self.clip_buttons_layout.addWidget(self.edit_button)
self.delete_button = QPushButton("Delete")
self.delete_button.clicked.connect(self.delete_clip)
self.delete_button.setEnabled(False)
self.clip_buttons_layout.addWidget(self.delete_button)
self.right_layout.addLayout(self.clip_buttons_layout)
# ---Add layouts to main_layout ---
self.main_layout.addLayout(self.left_layout, 1)
self.main_layout.addLayout(self.center_layout, 3)
self.main_layout.addLayout(self.right_layout, 1)
# Timer setup
self.timer.timeout.connect(self.update_waveform_cursor)
# Waveform zooming and double-click
view_box = self.waveform_plot.getViewBox()
view_box.setMouseEnabled(x=True, y=True)
view_box.setLimits(xMin=None, xMax=None, yMin=None, yMax=None)
def wheelEvent(event, axis=None):
    mouse_pos = view_box.mapSceneToView(event.pos())
    mouse_x, mouse_y = mouse_pos.x(), mouse_pos.y()
    x_range, y_range = view_box.viewRange()
    x_min, x_max = x_range[0], x_range[1]
    y_min, y_max = y_range[0], y_range[1]
    x_span = x_max - x_min
    y_span = y_max - y_min
    zoom_factor = 1.2 if event.delta() < 0 else 1 / 1.2
    new_x_span = x_span * zoom_factor

```



```

new_y_span = y_span * zoom_factor
new_x_span = max(new_x_span, 0.1)
new_y_span = max(new_y_span, 0.02)
x_ratio = (mouse_x - x_min) / x_span
new_x_min = mouse_x - new_x_span * x_ratio
new_x_max = mouse_x + new_x_span * (1 - x_ratio)
clamp_x_min, clamp_x_max = (self.start_time, self.end_time) if self.start_time and
self.end_time else (0, self.video_duration)
if new_x_span > (clamp_x_max - clamp_x_min):
    new_x_span = clamp_x_max - clamp_x_min
    new_x_min = clamp_x_min
    new_x_max = clamp_x_max
else:
    if new_x_min < clamp_x_min:
        new_x_min = clamp_x_min
        new_x_max = new_x_min + new_x_span
    if new_x_max > clamp_x_max:
        new_x_max = clamp_x_max
        new_x_min = new_x_max - new_x_span
y_ratio = (mouse_y - y_min) / y_span
new_y_min = mouse_y - new_y_span * y_ratio
new_y_max = mouse_y + new_y_span * (1 - y_ratio)
view_box.setRange(xRange=(new_x_min, new_x_max), yRange=(new_y_min,
new_y_max), padding=0)
def mouseDoubleClickEvent(event):
    if self.audio_waveform:
        x_min, x_max = (self.start_time, self.end_time) if self.start_time and self.end_time
else (0, self.video_duration)
_, samples = self.audio_waveform
if samples.size > 0:
    if self.start_time and self.end_time:
        time_axis, samples = self.audio_waveform
        sample_rate = len(time_axis) / self.video_duration if self.video_duration > 0 else 1
        start_idx = int(self.start_time * sample_rate)
        end_idx = int(self.end_time * sample_rate)
        start_idx = max(0, min(start_idx, len(samples) - 1))
        end_idx = max(start_idx, min(end_idx, len(samples)))
        if start_idx < end_idx:
            clip_samples = samples[start_idx:end_idx]
            max_amplitude = np.max(np.abs(clip_samples))
            max_amplitude = max(max_amplitude * 1.1, 0.1)
        else:
            max_amplitude = 0.1
    else:
        max_amplitude = np.max(np.abs(samples))
        max_amplitude = max(max_amplitude * 1.1, 0.1)

```

```

        y_min, y_max = -max_amplitude, max_amplitude
    else:
        y_min, y_max = -1, 1
    view_box.setRange(xRange=(x_min, x_max), yRange=(y_min, y_max), padding=0)
    view_box.wheelEvent = wheelEvent
    view_box.mouseDoubleClickEvent = mouseDoubleClickEvent
def setup_menu_bar(self):
    menu_bar = self.menuBar()
    file_menu = menu_bar.addMenu("&File")
    self.load_folder_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_DirOpenIcon), "&Load
Folder", self,
        shortcut="Ctrl+O", triggered=self.load_folder)
    file_menu.addAction(self.load_folder_action)
    exit_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_DialogCloseButton),
"E&xit", self,
        shortcut="Ctrl+Q", triggered=self.close_application)
    file_menu.addAction(exit_action)

    playback_menu = menu_bar.addMenu("&Playback")
    self.play_pause_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaPlay),
"&Play/Pause", self,
        shortcut="Space", triggered=self.toggle_play_pause)
    playback_menu.addAction(self.play_pause_action)
    if os.path.exists('assets/start_marker_icon.svg'):
        self.mark_start_action = QAction(QIcon('assets/start_marker_icon.svg'), "&Mark Start",
self,
            shortcut="Ctrl+[", triggered=self.mark_start)
    else:
        self.mark_start_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaSeekBackward),
"&Mark Start", self,
            shortcut="Ctrl+[", triggered=self.mark_start)
    playback_menu.addAction(self.mark_start_action)
    if os.path.exists('assets/end_marker_icon.svg'):
        self.mark_end_action = QAction(QIcon('assets/end_marker_icon.svg'), "&Mark End",
self,
            shortcut="Ctrl+]", triggered=self.mark_end)
    else:
        self.mark_end_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaSeekForward),
"&Mark End", self,
            shortcut="Ctrl+]", triggered=self.mark_end)
    playback_menu.addAction(self.mark_end_action)

```

```

        self.rotate_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_BrowserReload),
"&Rotate Video", self,
        shortcut="Ctrl+R", triggered=self.rotate_video)
        playback_menu.addAction(self.rotate_action)
        self.export_clip_action =
QAction(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_DialogSaveButton),
"&Export Clip", self,
        shortcut="Ctrl+S", triggered=self.save_clip_and_export)
        playback_menu.addAction(self.export_clip_action)
def setup_toolbar(self):
    self.toolbar.addAction(self.load_folder_action)
    self.toolbar.addAction(self.play_pause_action)
    self.play_pause_action.setEnabled(False)
    self.toolbar.addAction(self.rotate_action)
    self.rotate_action.setEnabled(False)
    if os.path.exists('assets/start_marker_icon.svg'):
        self.mark_start_action.setIcon(QIcon('assets/start_marker_icon.svg'))
    else:
        self.mark_start_action.setIcon(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaSeekBackward))
    self.toolbar.addAction(self.mark_start_action)
    self.mark_start_action.setEnabled(False)
    if os.path.exists('assets/end_marker_icon.svg'):
        self.mark_end_action.setIcon(QIcon('assets/end_marker_icon.svg'))
    else:
        self.mark_end_action.setIcon(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaSeekForward))
    self.toolbar.addAction(self.mark_end_action)
    self.mark_end_action.setEnabled(False)
    self.toolbar.addSeparator()
    self.video_label = QLabel("Exam Type:")
    self.toolbar.addWidget(self.video_label)
    self.video_dropdown = QComboBox()
    self.video_dropdown.addItem("")
    self.video_dropdown.addItem("Endoscopic Exam")
    self.video_dropdown.addItem("Acoustic Exam")
    self.video_dropdown.addItem("Audio Only")
    self.video_dropdown.currentTextChanged.connect(self.update_dropdown_states)
    self.toolbar.addWidget(self.video_dropdown)
    self.scope_label = QLabel("Scope Type:")
    self.scope_label.setStyleSheet("QLabel { color: black; }")
    self.toolbar.addWidget(self.scope_label)
    self.scope_dropdown = QComboBox()
    self.scope_dropdown.addItem("")
    self.scope_dropdown.addItem("Rigid")
    self.scope_dropdown.addItem("Flexible")
    self.scope_dropdown.setStyleSheet("QComboBox:disabled { color: gray; } QComboBox { color: black; }")
    self.toolbar.addWidget(self.scope_dropdown)
    self.light_label = QLabel("Light Type:")

```

```

self.light_label.setStyleSheet("QLabel { color: black; }")
self.toolbar.addWidget(self.light_label)
self.light_dropdown = QComboBox()
self.light_dropdown.addItem("", "Strobe", "Halogen"])
self.light_dropdown.setStyleSheet("QComboBox:disabled { color: gray; } QComboBox {
color: black; }")
self.toolbar.addWidget(self.light_dropdown)
self.speech_label = QLabel("Speech Type:")
self.speech_label.setStyleSheet("QLabel { color: black; }")
self.toolbar.addWidget(self.speech_label)
self.speech_dropdown = QComboBox()
self.speech_dropdown.addItem(self.speech_types + ["<new>"])
self.speech_dropdown.currentTextChanged.connect(self.handle_speech_type_selection)
self.speech_dropdown.setStyleSheet("QComboBox:disabled { color: gray; } QComboBox {
color: black; }")
self.toolbar.addWidget(self.speech_dropdown)
self.toolbar.addSeparator()
self.toolbar.addAction(self.export_clip_action)
self.export_clip_action.setEnabled(False)
def load_speech_types(self):
    default_speech_types = ["", "Vowel - ehh", "Vowel - ahh", "Sentences", "Rainbow
Passage"]
    try:
        if os.path.exists(self.global_config_file):
            with open(self.global_config_file, 'r') as f:
                data = json.load(f)
                return data.get("speech_types", default_speech_types)
        else:
            return default_speech_types
    except (json.JSONDecodeError, IOError):
        return default_speech_types

def save_speech_types(self):
    try:
        with open(self.global_config_file, 'w') as f:
            json.dump({"speech_types": self.speech_types}, f, indent=4)
    except IOError as e:
        QMessageBox.warning(self, "Error", f"Failed to save speech types: {str(e)}")

def load_user_config(self):
    default_config = {"default_folder": str(Path.home())}
    try:
        if self.user_config_file.exists():
            with open(self.user_config_file, 'r') as f:
                data = json.load(f)
                return data

```

```

        else:
            return default_config
    except (json.JSONDecodeError, IOError):
        return default_config

def save_user_config(self, folder_path):
    config = {"default_folder": folder_path}
    try:
        with open(self.user_config_file, 'w') as f:
            json.dump(config, f, indent=4)
    except IOError as e:
        QMessageBox.warning(self, "Error", f"Failed to save user config: {str(e)}")

def load_default_folder(self):
    config = self.load_user_config()
    default_folder = config.get("default_folder")
    try:
        if default_folder and os.path.isdir(default_folder):
            self.folder = default_folder
            self.trimmed_folder = os.path.join(self.folder, "TrimmedClips")
            os.makedirs(self.trimmed_folder, exist_ok=True)
            self.file_list = [f for f in os.listdir(self.folder) if f.endswith(".mp4") and
os.path.isfile(os.path.join(self.folder, f))]
            if self.file_list:
                self.file_list_widget.clear()
                for file_name in self.file_list:
                    item = QListWidgetItem(file_name)
                    item.setData(Qt.ItemDataRole.UserRole, file_name) # Updated to
ItemDataRole.UserRole
                    self.file_list_widget.addItem(item)
            else:
                print(f"No .mp4 files found in default folder: {self.folder}")
        else:
            print(f"Default folder not set or invalid: {default_folder}")
    except (PermissionError, OSError) as e:
        QMessageBox.warning(self, "Folder Error", f"Cannot access default folder: {str(e)}")

def load_folder(self):
    folder = QFileDialog.getExistingDirectory(self, "Select Folder",
self.load_user_config().get("default_folder", ""))
    if folder:
        try:
            self.folder = folder
            self.trimmed_folder = os.path.join(self.folder, "TrimmedClips")
            os.makedirs(self.trimmed_folder, exist_ok=True)
            self.file_list = [f for f in os.listdir(self.folder) if f.endswith(".mp4") and
os.path.isfile(os.path.join(self.folder, f))]
            self.file_list_widget.clear()

```

```

        for file_name in self.file_list:
            item = QListWidgetItem(file_name)
            item.setData(Qt.ItemDataRole.UserRole, file_name) # Updated to
ItemDataRole.UserRole
            self.file_list_widget.addItem(item)
            self.save_user_config(folder)
            self.load_all_annotations()
            print(f"Default folder set to: {folder}")
        except (PermissionError, OSError) as e:
            QMessageBox.warning(self, "Folder Error", f"Cannot access folder: {str(e)}")
def load_all_annotations(self):
    self.annotations = []
    annotations_file = os.path.join(self.folder, "annotations.xlsx")
    if os.path.exists(annotations_file):
        try:
            df = pd.read_excel(annotations_file)
            for index, row in df.iterrows():
                clip_info = [
                    row["Start Time"], row["End Time"], row["Exam Type"],
                    row["Scope Type"], row["Light Type"], row["Speech Type"],
                    row["Clip File"], row["Source File"]
                ]
                self.annotations.append(clip_info)
            print(f"Loaded {len(self.annotations)} annotations from {annotations_file}")
        except Exception as e:
            print(f"Error loading annotations: {str(e)}")
    else:
        print(f"No annotations file found at: {annotations_file}")
def load_clips(self):
    self.clip_list.clear()
    if not self.current_file:
        print("No current file selected, cannot load clips.")
        return
    current_source_file = os.path.basename(self.current_file)
    print(f>Loading clips for source file: {current_source_file}")
    print(f>Total annotations: {len(self.annotations)}")
    for index, clip in enumerate(self.annotations):
        if clip[7] == current_source_file:
            item = QListWidgetItem(clip[6])
            item.setData(Qt.ItemDataRole.UserRole, index) # Updated to ItemDataRole.UserRole
            self.clip_list.addItem(item)
            print(f"Added clip: {clip[6]}")
def select_clip(self, item):
    index = item.data(Qt.ItemDataRole.UserRole) # Updated to ItemDataRole.UserRole
    clip = self.annotations[index]
    self.start_time = clip[0]

```

```

self.end_time = clip[1]
self.video_dropdown.setCurrentText(clip[2])
self.scope_dropdown.setCurrentText(clip[3])
self.light_dropdown.setCurrentText(clip[4])
self.speech_dropdown.setCurrentText(clip[5])
self.start_marker.setPos(self.start_time)
self.start_marker.setVisible(True)
self.end_marker.setPos(self.end_time)
self.end_marker.setVisible(True)
self.player.setPosition(int(self.start_time * 1000))
self.edit_button.setEnabled(True)
self.delete_button.setEnabled(True)

if self.audio_waveform:
    self.waveform_plot.setXRange(self.start_time, self.end_time, padding=0)
    time_axis, samples = self.audio_waveform
    sample_rate = len(time_axis) / self.video_duration if self.video_duration > 0 else 1
    start_idx = int(self.start_time * sample_rate)
    end_idx = int(self.end_time * sample_rate)
    start_idx = max(0, min(start_idx, len(samples) - 1))
    end_idx = max(start_idx, min(end_idx, len(samples)))
    if start_idx < end_idx:
        clip_samples = samples[start_idx:end_idx]
        max_amplitude = np.max(np.abs(clip_samples))
        max_amplitude = max(max_amplitude * 1.1, 0.1)
        self.waveform_plot.setYRange(-max_amplitude, max_amplitude, padding=0)
    else:
        self.waveform_plot.setYRange(-0.1, 0.1, padding=0)

def edit_clip(self):
    selected_items = self.clip_list.selectedItems()
    if not selected_items:
        return
    item = selected_items[0]
    index = item.data(Qt.ItemDataRole.UserRole) # Updated to ItemDataRole.UserRole
    clip = self.annotations[index]
    dialog = QDialog(self)
    dialog.setWindowTitle("Edit Clip")
    layout = QGridLayout(dialog)
    layout.addWidget(QLabel("Start Time:"), 0, 0)
    start_input = QLineEdit(str(clip[0]))
    start_input.setReadOnly(True)
    layout.addWidget(start_input, 0, 1)
    layout.addWidget(QLabel("End Time:"), 1, 0)
    end_input = QLineEdit(str(clip[1]))
    end_input.setReadOnly(True)
    layout.addWidget(end_input, 1, 1)

```

```

layout.addWidget(QLabel("Exam Type:"), 2, 0)
video_edit = QComboBox()
video_edit.addItem("")
video_edit.addItem("Endoscopic Exam")
video_edit.addItem("Acoustic Exam")
video_edit.addItem("Audio Only")
video_edit.setCurrentText(clip[2])
layout.addWidget(video_edit, 2, 1)
layout.addWidget(QLabel("Scope Type:"), 3, 0)
scope_edit = QComboBox()
scope_edit.addItem("")
scope_edit.addItem("Rigid")
scope_edit.addItem("Flexible")
scope_edit.setCurrentText(clip[3])
layout.addWidget(scope_edit, 3, 1)
layout.addWidget(QLabel("Light Type:"), 4, 0)
light_edit = QComboBox()
light_edit.addItem("")
light_edit.addItem("Strobe")
light_edit.addItem("Halogen")
light_edit.setCurrentText(clip[4])
layout.addWidget(light_edit, 4, 1)
layout.addWidget(QLabel("Speech Type:"), 5, 0)
speech_edit = QComboBox()
speech_edit.addItem(self.speech_types + ["<new>"])
speech_edit.setCurrentText(clip[5])
layout.addWidget(speech_edit, 5, 1)
save_btn = QPushButton("Save")
save_btn.clicked.connect(lambda: self.save_edited_clip(dialog, index, start_input.text(),
end_input.text(),
                                video_edit.currentText(), scope_edit.currentText(),
                                light_edit.currentText(), speech_edit.currentText()))
layout.addWidget(save_btn, 6, 0)
cancel_btn = QPushButton("Cancel")
cancel_btn.clicked.connect(dialog.reject)
layout.addWidget(cancel_btn, 6, 1)
dialog.exec()
def save_edited_clip(self, dialog, index, start_str, end_str, video, scope, light, speech):
    try:
        clip = self.annotations[index]
        start_time = clip[0]
        end_time = clip[1]
        clip[0] = start_time
        clip[1] = end_time
        clip[2] = video
        clip[3] = scope
        clip[4] = light
        clip[5] = speech
        self.export_annotations()
        self.load_clips()
        dialog.accept()
    except ValueError:

```



```

        QMessageBox.warning(self, "Invalid Input", "An unexpected error occurred.")
def delete_clip(self):
    selected_items = self.clip_list.selectedItems()
    if not selected_items:
        return
    item = selected_items[0]
    index = item.data(Qt.ItemDataRole.UserRole) # Updated to ItemDataRole.UserRole
    clip_file = self.annotations[index][6]
    exam_type = self.annotations[index][2]
    reply = QMessageBox.question(self, "Confirm Delete", f"Delete clip {clip_file}?",
                                QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No)
    if reply == QMessageBox.StandardButton.Yes:
        extension = ".wav" if exam_type == "Audio Only" else ".mp4"
        clip_path = os.path.join(self.trimmed_folder, f"{clip_file}{extension}")
        try:
            if os.path.exists(clip_path):
                os.remove(clip_path)
                print(f"Deleted file: {clip_path}")
            else:
                print(f"File not found, removing from annotations anyway: {clip_path}")
        except Exception as e:
            print(f"Error deleting file {clip_path}: {str(e)}")

        del self.annotations[index]
        self.export_annotations()
        self.load_clips()
        self.start_marker.setVisible(False)
        self.end_marker.setVisible(False)
        self.edit_button.setEnabled(False)
        self.delete_button.setEnabled(False)
def handle_speech_type_selection(self, selected_text):
    if selected_text == "<new>":
        new_speech_type, ok = QDialog.getText(self, "New Speech Type", "Enter a new
speech type:")
        if ok and new_speech_type.strip():
            if new_speech_type not in self.speech_types:
                self.speech_types.append(new_speech_type)
                self.save_speech_types()
                self.update_speech_dropdown(new_speech_type)
            else:
                QMessageBox.warning(self, "Duplicate", "This speech type already exists.")
        else:
            self.speech_dropdown.setCurrentIndex(0)
    elif selected_text:
        print(f"Selected speech type: {selected_text}")

```

```

def update_speech_dropdown(self, new_selection):
    self.speech_dropdown.blockSignals(True)
    self.speech_dropdown.clear()
    self.speech_dropdown.addItem(self.speech_types + ["<new>"])
    self.speech_dropdown.setCurrentText(new_selection)
    self.speech_dropdown.blockSignals(False)
def update_dropdown_states(self, selected_text):
    if selected_text in ["Acoustic Exam", "Audio Only"]:
        self.scope_dropdown.setEnabled(False)
        self.light_dropdown.setEnabled(False)
        self.scope_dropdown.setStyleSheet("QComboBox:disabled { color: gray; } QComboBox
{ color: black; }")
        self.light_dropdown.setStyleSheet("QComboBox:disabled { color: gray; } QComboBox
{ color: black; }")
        self.speech_dropdown.setEnabled(True)
        self.speech_dropdown.setStyleSheet("QComboBox { color: black; }")
        self.scope_label.setStyleSheet("QLabel { color: gray; }")
        self.light_label.setStyleSheet("QLabel { color: gray; }")
        self.speech_label.setStyleSheet("QLabel { color: black; }")
    else:
        self.scope_dropdown.setEnabled(True)
        self.light_dropdown.setEnabled(True)
        self.speech_dropdown.setEnabled(True)
        self.scope_dropdown.setStyleSheet("QComboBox { color: black; }")
        self.light_dropdown.setStyleSheet("QComboBox { color: black; }")
        self.speech_dropdown.setStyleSheet("QComboBox { color: black; }")
        self.scope_label.setStyleSheet("QLabel { color: black; }")
        self.light_label.setStyleSheet("QLabel { color: black; }")
        self.speech_label.setStyleSheet("QLabel { color: black; }")
def update_start_time(self):
    self.start_time = max(0, self.start_marker.value())
    print(f"Start time updated: {self.start_time:.2f} seconds")
    self.player.setPosition(int(self.start_time * 1000))
def update_end_time(self):
    self.end_time = max(self.start_time if self.start_time is not None else 0,
self.end_marker.value())
    print(f"End time updated: {self.end_time:.2f} seconds")
    self.player.setPosition(int(self.end_time * 1000))
def select_file(self, item):
    if item is not None:
        file_name = item.data(Qt.ItemDataRole.UserRole) # Updated to ItemDataRole.UserRole
        print(f"Selected file: {file_name}")
        self.player.stop()
        self.current_file = os.path.join(self.folder, file_name)
        print(f"Current file set to: {self.current_file}")
        self.play_pause_action.setEnabled(True)

```

```

self.mark_start_action.setEnabled(True)
self.mark_end_action.setEnabled(True)
self.export_clip_action.setEnabled(True)
self.rotate_action.setEnabled(True)
self.start_marker.setVisible(False)
self.end_marker.setVisible(False)
self.start_time = None
self.end_time = None
self.load_clips()

self.player.setSource(QUrl.fromLocalFile(self.current_file))
self.player.durationChanged.connect(self.set_video_duration)
self.extract_audio_waveform()
if self.audio_waveform:
    self.waveform_plot.setXRange(0, self.video_duration, padding=0)
    _, samples = self.audio_waveform
    if samples.size > 0:
        max_amplitude = np.max(np.abs(samples))
        max_amplitude = max(max_amplitude * 1.1, 0.1)
        self.waveform_plot.setYRange(-max_amplitude, max_amplitude, padding=0)
    else:
        self.waveform_plot.setYRange(-1, 1, padding=0)
def set_video_duration(self, duration):
    self.video_duration = duration / 1000.0
    print(f"Video duration set to: {self.video_duration:.2f} seconds")
def toggle_play_pause(self):
    if not self.current_file:
        QMessageBox.warning(self, "No File", "Please select a file to play.")
        return
    if self.player.playbackState() == QMediaPlayer.PlaybackState.PlayingState:
        self.player.pause()
        self.timer.stop()
        self.play_pause_action.setIcon(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaPlay))
        self.play_pause_action.setText("Play")
    else:
        self.player.play()
        self.timer.start(30)
        self.play_pause_action.setIcon(QApplication.style().standardIcon(QStyle.StandardPixmap.SP_MediaPause))
        self.play_pause_action.setText("Pause")
def update_waveform_cursor(self):
    current_time = self.player.position() / 1000.0
    self.cursor.setPos(current_time)
    if self.player.playbackState() == QMediaPlayer.PlaybackState.StoppedState:
        self.timer.stop()

```

```

def extract_audio_waveform(self):
    if not self.current_file:
        print("No current file to extract waveform.")
        return
    print(f"Extracting waveform for: {self.current_file}")
    try:
        stream = ffmpeg.input(self.current_file)
        stream = stream.output('pipe:', format='wav', acodec='pcm_s16le', loglevel="quiet")
        out, _ = ffmpeg.run(stream, capture_stdout=True)
        audio = AudioSegment.from_file(io.BytesIO(out), format="wav")
        samples = np.array(audio.get_array_of_samples())
        samples = samples.astype(np.float32) / np.iinfo(audio.array_type).max
        audio_duration = len(samples) / audio.frame_rate
        if abs(audio_duration - self.video_duration) > 0.1:
            print(f"Warning: Audio duration ({audio_duration:.2f}s) differs from video
({self.video_duration:.2f}s)")
            time_axis = np.linspace(0, self.video_duration, num=len(samples))
            print(f"Waveform samples length: {len(samples)}, time_axis length: {len(time_axis)}")
            self.waveform_curve.setData(time_axis, samples)
            self.audio_waveform = (time_axis, samples)
            self.waveform_plot.setXRange(0, self.video_duration, padding=0)
            if samples.size > 0:
                max_amplitude = np.max(np.abs(samples))
                max_amplitude = max(max_amplitude * 1.1, 0.1)
                self.waveform_plot.setYRange(-max_amplitude, max_amplitude, padding=0)
            else:
                self.waveform_plot.setYRange(-1, 1, padding=0)
    except ffmpeg.Error as e:
        QMessageBox.critical(self, "FFmpeg Error", f"Error extracting audio:
{str(e.stderr.decode() if e.stderr else e)}")
    except Exception as e:
        QMessageBox.critical(self, "Waveform Error", f"Unexpected error: {str(e)}")
def mark_start(self):
    self.start_time = self.player.position() / 1000.0
    if self.start_time is not None:
        print(f"Start marked: {self.start_time:.2f} seconds")
        self.start_marker.setVisible(True)
        self.start_marker.setPos(self.start_time)
def mark_end(self):
    self.end_time = self.player.position() / 1000.0
    if self.end_time is not None:
        print(f"End marked: {self.end_time:.2f} seconds")
        self.end_marker.setVisible(True)
        self.end_marker.setPos(self.end_time)
def save_clip_and_export(self):

```

```

if self.start_time is None or self.end_time is None or self.start_time >= self.end_time:
    QMessageBox.warning(self, "Invalid Selection", "Please select valid start and end
points.")
    return
file_path = os.path.join(self.folder, self.current_file)
source_file = os.path.basename(self.current_file)
video = self.video_dropdown.currentText()
scope = self.scope_dropdown.currentText()
light = self.light_dropdown.currentText()
speech = self.speech_dropdown.currentText()
try:
    source_base = os.path.splitext(source_file)[0]
    if video == "Audio Only":
        clip_name_base = f"audio_{source_base}_{self.start_time:.2f}_{self.end_time:.2f}"
        output_file = os.path.join(self.trimmed_folder, f"{clip_name_base}.wav")
        stream = ffmpeg.input(file_path, ss=self.start_time, t=self.end_time - self.start_time)
        stream = ffmpeg.output(stream, output_file, format='wav', acodec='pcm_s16le',
loglevel="quiet")
        ffmpeg.run(stream, overwrite_output=True)
        QMessageBox.information(self, "Success", f"Audio clip saved to TrimmedClips as
{clip_name_base}")
    else:
        clip_name_base = f"clip_{source_base}_{self.start_time:.2f}_{self.end_time:.2f}"
        output_file = os.path.join(self.trimmed_folder, f"{clip_name_base}.mp4")
        stream = ffmpeg.input(file_path, ss=self.start_time, t=self.end_time - self.start_time)
        stream = ffmpeg.output(stream, output_file, c='copy', loglevel="quiet")
        ffmpeg.run(stream, overwrite_output=True)
        QMessageBox.information(self, "Success", f"Video clip saved to TrimmedClips as
{clip_name_base}")
        self.annotations.append([self.start_time, self.end_time, video, scope, light, speech,
clip_name_base, source_file])
        self.export_annotations()
        self.load_clips()
    except ffmpeg.Error as e:
        QMessageBox.critical(self, "Error", f"Failed to save clip: {str(e.stderr.decode() if e.stderr
else e)}")
    except Exception as e:
        QMessageBox.critical(self, "Error", f"An unexpected error occurred: {str(e)}")
def export_annotations(self):
    try:
        df = pd.DataFrame(self.annotations, columns=[
            "Start Time", "End Time", "Exam Type", "Scope Type", "Light Type", "Speech Type",
"Clip File", "Source File"
        ])
        df.to_excel(os.path.join(self.folder, "annotations.xlsx"), index=False)
        print(f"Annotations updated in {os.path.join(self.folder, 'annotations.xlsx')}")

```

```

    except Exception as e:
        QMessageBox.critical(self, "Error", f"Failed to update annotations.xlsx: {str(e)}")
def close_application(self):
    self.timer.stop()
    self.player.stop()
    QApplication.quit()
def rotate_video(self):
    self.rotation_angle = (self.rotation_angle + 90) % 360
    # Note: QMediaPlayer doesn't support rotation natively
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = LiveAnnotationTool()
    window.show()
    sys.exit(app.exec())

```